

Київський національний університет імені Тараса Шевченка



ЗВІТ

до лабораторної роботи №8

З дисципліни БАЗИ ДАНИХ ТА ІНФОРМАЦІЙНІ СИСТЕМИ

Виконав
Студент 3 курсу
Групи МІТ-31
Факультету Інформаційних Технологій
Лаптев Олександр

Київ, 2026

Тема: Розширені можливості Redis

Мета: Закріпити знання про роботу з Redis та ознайомитися з розширеним функціоналом - транзакціями, скриптами на Lua, публікацією/підпискою (Pub/Sub) та потоками Redis Streams.

Вступ

У цій лабораторній роботі досліджуються розширені можливості In-Memory бази даних Redis. Виконано практичні завдання з використання транзакцій для безпечного оновлення даних, виконання Lua-скриптів на стороні сервера, налаштування обміну повідомленнями через Pub/Sub та роботи з потоками даних Redis Streams. Усі практичні дії виконувалися в ізолюваному середовищі Docker за допомогою інтерфейсу redis-cli.

Хід роботи :

Транзакції у Redis Для безпечного додавання кількох ключів та імітації конкурентного доступу виконано наступні команди:

```
127.0.0.1:6379> set purchases 5
OK
127.0.0.1:6379> watch balance
OK
127.0.0.1:6379> multi
OK
127.0.0.1:6379(TX)> decrby
(error) ERR wrong number of arguments for 'decrby' command
127.0.0.1:6379(TX)> decrby balance 100
QUEUED
127.0.0.1:6379(TX)> incr purchases
QUEUED
127.0.0.1:6379(TX)> exec
(error) EXECABORT Transaction discarded because of previous errors.
127.0.0.1:6379> multi
OK
127.0.0.1:6379(TX)> decrby balance 100
QUEUED
127.0.0.1:6379(TX)> incr purchases
QUEUED
127.0.0.1:6379(TX)> exec
1) (integer) 900
2) (integer) 6
127.0.0.1:6379> |
```

Рис. 1. Після вводу MULTI відповіді на команди будуть QUEUED. Лише після EXEC вони виконуються реально.

Луа-скрипти в Redis Для перевірки наявності ключа та його створення (якщо він не існує) використано атомарну команду EVAL:

```
127.0.0.1:6379> eval "if redis.call('exists',KEYS[1]) == 0 then return redis.call ('set', KEYS[1],ARGV[1]) else return 'exists' end" 1 testkey "value"
OK
```

Рис. 2. Виконання команди EVAL "if redis.call('exists', KEYS[1]) == 0 then return redis.call('set', KEYS[1], ARGV[1]) else return 'exists' end" 1 testkey "value"

Цифра 1 вказує, що ми передаємо один ключ (testkey), а "value" — це аргумент.

Механізм Pub/Sub Для тестування відкрито два незалежні підключення до контейнера. У першому терміналі запущено підписку на канал:

SUBSCRIBE news

```
PS C:\Users\igarl> docker exec -it redis-container redis-cli
127.0.0.1:6379> SUBSCRIBE news
1) "subscribe"
2) "news"
3) (integer) 1
127.0.0.1:6379(subscribed mode)>
1) "message"
2) "news"
3) "Redis is awesome!"
Reading messages... (press Ctrl-C to quit or any key to type command)
```

Рис 3. Виконання команд Subscribe news та отримання відповіді від публішера

У другому терміналі виконано публікацію повідомлення:

```
PS C:\Users\igarl> docker exec -it redis-container redis-cli
127.0.0.1:6379> PUBLISH news "Redis is awesome!"
(integer) 1
127.0.0.1:6379> |
```

Рис 4. Виконання команди PUBLISH news

У першому терміналі було успішно отримано та відображено надіслане повідомлення.

Redis Streams (потіки даних) Створено потік mystream та додано до нього подію телеметрії:

```
127.0.0.1:6379> XADD mystream * sensor-id 1234 temperature 19.8
"1781182205198-0"
127.0.0.1:6379> XRANGE mystream - +
1) 1) "1781182205198-0"
    2) 1) "sensor-id"
        2) "1234"
        3) "temperature"
        4) "19.8"
127.0.0.1:6379> XREAD COUNT 2 STREAMS mystream 0
1) 1) "mystream"
    2) 1) 1) "1781182205198-0"
        2) 1) "sensor-id"
            2) "1234"
            3) "temperature"
            4) "19.8"
127.0.0.1:6379> |
```

Рис 5. Виконання команди XADD mystream * sensor-id 1234 temperature 19.8

Прочитано історію подій з потоку:

XRANGE mystream - +

Запущено споживача потоку для зчитування нових повідомлень:

XREAD COUNT 2 STREAMS mystream 0

Код програми з використанням Redis Streams

```
Lab_8 > red_streams.py > ...
1 import redis
2
3 # Підключення до локального контейнера Redis
4 client = redis.Redis(host='localhost', port=6379, decode_responses=True)
5
6 # 1. Додавання нових даних у потік (логіка XADD)
7 stream_name = 'mystream'
8 message_data = {'sensor-id': '9999', 'temperature': '22.5', 'status': 'active'}
9 message_id = client.xadd(stream_name, message_data)
10 print(f"Дані успішно додано до потоку. ID повідомлення: {message_id}")
11
12 # 2. Зчитування нових повідомлень зі стріму (логіка XREAD)
13 # Читаємо 2 останні повідомлення, починаючи з самого початку ('0')
14 messages = client.xread({stream_name: '0'}, count=2)
15
16 print("Отримані дані з потоку:")
17 for stream, data_list in messages:
18     for message_id, data in data_list:
19         print(f"ID: {message_id} -> Значення: {data}")
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```
PS C:\Users\igarl\Labs\Labs_DataBase> & C:\Users\igarl\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/igarl/Labs/Labs_DataBase/Lab_7/redis_test.py
Поточне значення лічильника: 1
Список задач: ['Write report']
Повідомлення опубліковано у канал 'updates'
PS C:\Users\igarl\Labs\Labs_DataBase> & C:\Users\igarl\AppData\Local\Python\pythoncore-3.14-64\python.exe c:/Users/igarl/Labs/Labs_DataBase/Lab_8/red_streams.py
Дані успішно додано до потоку. ID повідомлення: 1781182986107-0
Отримані дані з потоку:
ID: 1781182205198-0 -> Значення: {'sensor-id': '1234', 'temperature': '19.8'}
ID: 1781182986107-0 -> Значення: {'sensor-id': '9999', 'temperature': '22.5', 'status': 'active'}
PS C:\Users\igarl\Labs\Labs_DataBase>
```

Рис.6 Виконання коду програми

Теоретичні висновки

- Redis Pub/Sub ідеально підходить для систем реального часу, таких як чати або системи миттєвих сповіщень, де повідомлення потрібно доставити активним користувачам миттєво.
- Redis Streams є надійнішим механізмом для великих розподілених систем, оскільки ця структура зберігає історію повідомлень. Це дозволяє ефективно використовувати її для збору логів, телеметрії з IoT-датчиків та надійних черг завдань.
- Плюси Redis як брокера повідомлень: надзвичайна швидкість читання та запису в оперативній пам'яті, простота налаштування конфігурації.
- Мінуси Redis як брокера повідомлень: обмеженість загальним обсягом оперативної пам'яті сервера, менша кількість складних механізмів маршрутизації порівняно зі спеціалізованими рішеннями, а також повна втрата даних у Pub/Sub, якщо підписник був офлайн у момент публікації.

Відповіді на запитання для самоперевірки

1. Як працюють транзакції в Redis і для чого потрібна команда WATCH? Транзакції (команди MULTI та EXEC) групують операції для їх послідовного виконання без втручання інших клієнтів. Команда WATCH забезпечує оптимістичне блокування: якщо інший клієнт змінить ключ, за яким ведеться спостереження, до виклику EXEC, вся транзакція буде скасована.
2. Для чого використовують Lua-скрипти в Redis? Скрипти дозволяють виконувати складну логіку з кількома ключами безпосередньо на сервері як одну атомарну операцію. Це гарантує цілісність даних та значно зменшує мережеві затримки.
3. У чому суть моделі Pub/Sub і які її обмеження? Це патерн комунікації, де видавець відправляє дані у канал, а всі підключені до нього підписники їх отримують. Головне обмеження полягає у відсутності збереження стану (fire-and-forget): повідомлення назавжди зникає, якщо на момент відправки канал не мав активних підписників.
4. Що таке Redis Streams? Як зчитуються події зі стріму? Це структура даних, що працює як лог, до якого можна лише додавати записи у хронологічному порядку. На відміну від Pub/Sub, дані зберігаються постійно. Зчитування відбувається командами XRANGE (читання історії за діапазоном) або XREAD (прослуховування нових подій).
5. У яких випадках Redis може бути альтернативою через повідомлень (наприклад, RabbitMQ)? Redis стає кращим вибором, коли критично важливою є максимальна пропускна здатність та мінімальна затримка, а вимоги до складної маршрутизації повідомлень (наприклад, topic exchanges) є другорядними.

